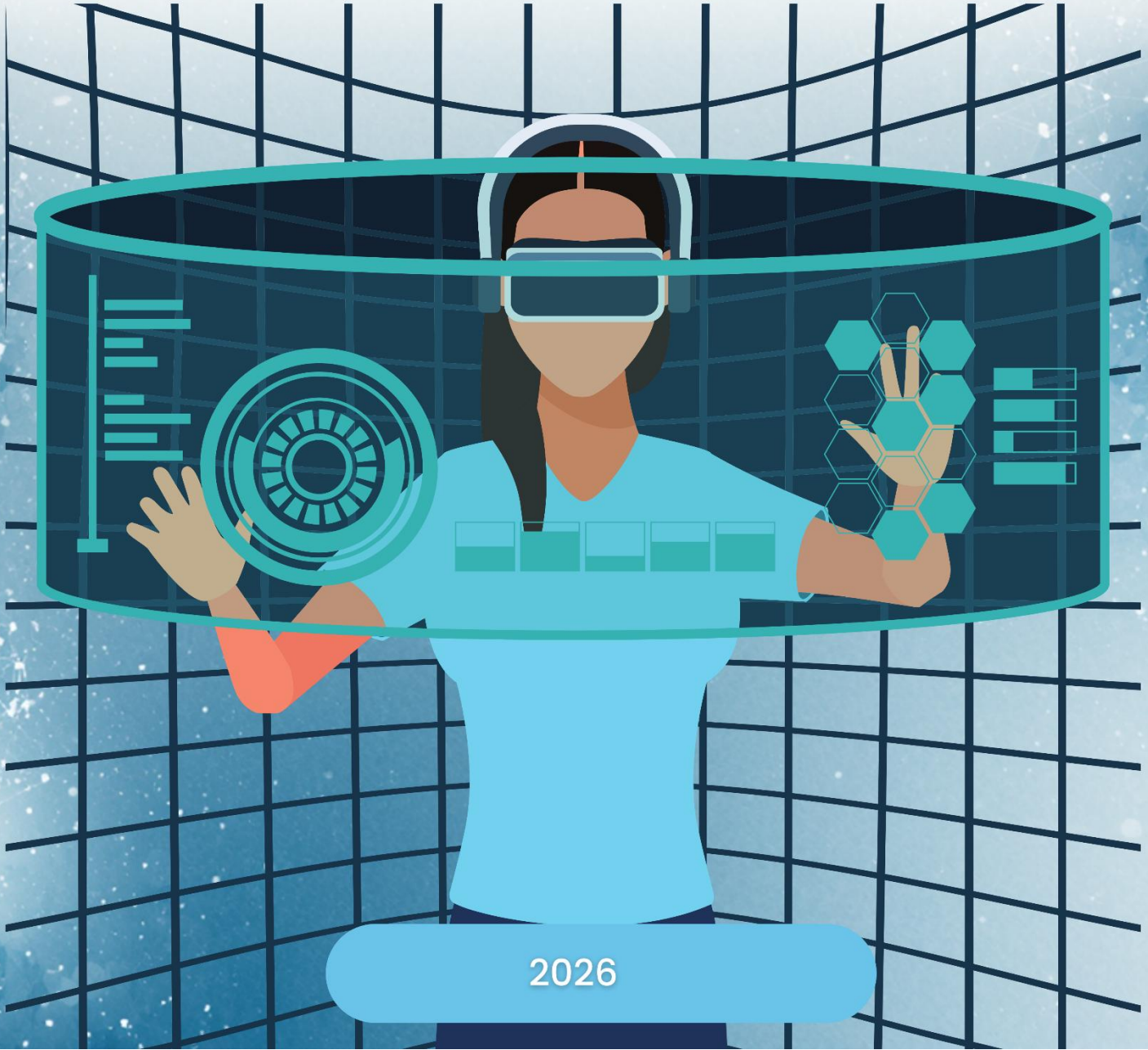
MODUL LKS

PROVINSI JAWA TIMUR

CLOUD COMPUTING



2026

1. Project Description

1.1. Background

In the cloud-native landscape of 2026, enterprises are no longer focused solely on rapid feature delivery; they also demand operational resilience, observability, and controlled scalability. This project designs an event-driven, serverless order processing system that balances speed and reliability through a loosely coupled architecture. By leveraging services such as AWS Lambda for compute, Amazon EventBridge for event routing, and Amazon SQS for distributed message queuing, the system processes each incoming order asynchronously and independently. Every stage—validation, payment processing, notification, and logging—is triggered by events rather than direct service-to-service calls, significantly reducing tight coupling and improving fault isolation. This event-driven paradigm ensures elasticity under unpredictable workloads while maintaining high availability and graceful degradation in case of component failures.

To guarantee consistency and repeatability across environments, infrastructure provisioning is fully automated using AWS CloudFormation as an Infrastructure as Code (IaC) solution. All cloud resources—including Lambda functions, IAM roles, API configurations, and monitoring components—are defined declaratively and version-controlled. Deployment is orchestrated through a GitOps-based CI/CD pipeline powered by GitHub Actions, enabling automated build, test, and deployment workflows directly from the repository. Application releases follow a Blue/Green deployment strategy implemented with AWS CodeDeploy, allowing seamless traffic shifting between environments and automated rollback in case health checks fail. The integration of automated provisioning, progressive delivery, and GitOps practices establishes a robust, scalable, and production-ready architecture tailored to the operational demands of modern enterprise systems.

1.2. Task

As a Cloud Engineer, you are responsible for building an enterprise-grade serverless order management system with the following specifications:

TASK LIST:

1. Log in to the AWS Management Console using the provided classroom account,
2. Provision the infrastructure using Infrastructure as Code with AWS CloudFormation,
3. Configure the networking layer: VPC, subnets, security groups, and VPC endpoints.
4. Set up Amazon S3 buckets for orders, logs, and Terraform state storage.
5. Configure secure configuration management using AWS Secrets Manager and AWS Systems Manager Parameter Store.
6. Deploy seven AWS Lambda functions along with a shared Lambda layer.
7. Provision Amazon RDS PostgreSQL 15 and initialize the database schema.
8. Create a complete state machine using AWS Step Functions with full Amazon States Language (ASL) definition.
9. Configure three rules in Amazon EventBridge for event routing.
10. Configure a REST API using Amazon API Gateway, including stages and a Usage Plan.

11. Set up Blue/Green Lambda deployment using AWS CodeDeploy.
12. Implement a CI/CD pipeline with six jobs using GitHub Actions (build, test, security scan, package, deploy, verification).
13. Deploy the frontend using AWS Amplify and configure environment variables.
14. Set up monitoring and observability with Amazon CloudWatch Alarms, Dashboards, and AWS X-Ray tracing.
15. Perform end-to-end verification with Step Function.

1.3. Technical Details

1. Default Region: Virginia (us-east-1)
2. Service Prefix: All service names must start with techno
3. Source Code Repository: https://github.com/handipradana/serverprov.git
4. EC2 Operating System (if required): Amazon Linux 2023
5. Lambda Runtime: Python 3.11 on AWS Lambda
6. Database Engine: PostgreSQL 15.14 on Amazon RDS
7. Infrastructure as Code (IaC): AWS CloudFormation
8. Deployment Strategy: Blue/Green via AWS CodeDeploy (LambdaCanary10Percent5Minutes)
9. CI/CD Pipeline: GitHub Actions (.github/workflows/deploy.yml)
10. Observability: AWS X-Ray + Amazon CloudWatch Dashboard + Structured Logging

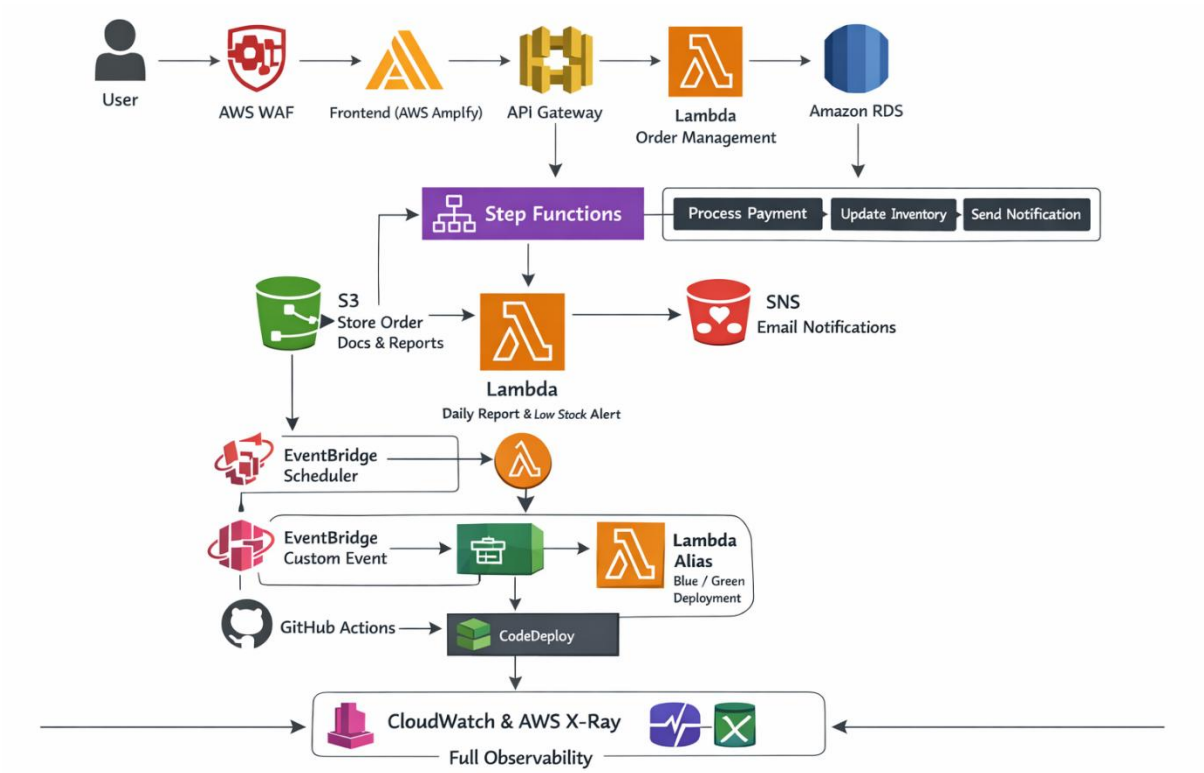
1.4. References

1. VPC: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html
2. Lambda: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
3. Lambda Traffic Shifting: https://docs.aws.amazon.com/lambda/latest/dg/configuration-aliases.html
4. RDS: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html
5. Step Functions: https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html
6. EventBridge: https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html
7. API Gateway: https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html
8. Amplify: https://docs.aws.amazon.com/amplify/latest/userguide/welcome.html
9. S3: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html
10. Secrets Manager: https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html
11. X-Ray: https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.
12. CodeDeploy User Guide: https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html

13. CodeDeploy Lambda Deployment: https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-steps-lambda.html
14. AppSpec File Reference: https://docs.aws.amazon.com/codedeploy/latest/userguide/reference-appspec-file-structure-resources.html
15. Deployment Configurations: https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-configurations.
16. GitHub Actions Quickstart: https://docs.github.com/en/actions/get-started/quickstart
17. GitHub Personal Access Token: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens
18. aws-actions/configure-aws-credentials: https://github.com/aws-actions/configure-aws-credentials

2. Architecture Diagram

The system architecture consists of 8 main flows:



3. Implementation using CloudFormation

Use CloudFormation to provision all infrastructure components. The entire environment must be defined as Infrastructure as Code (IaC).

3.1 Networking and Security

1. Virtual Private Cloud (VPC)

The network foundation is built using Amazon VPC with the name techno-vpc-serverless. The VPC is configured with an IPv4 CIDR block of 35.10.0.0/18 and an Amazon-provided IPv6 /56 block to enable dual-stack networking. This setup ensures scalability, future-proof IP addressing, and compatibility with modern IPv6-enabled services.

Two public subnets are provisioned across multiple Availability Zones to ensure high availability: techno-subnet-public-1 (35.10.0.0/25) in us-east-1a and techno-subnet-public-2 (35.10.1.0/25) in us-east-1b. These subnets are designed to host publicly accessible resources such as NAT Gateways or load balancers if required. In addition, two private subnets are configured for internal workloads: techno-subnet-private-1 (35.10.11.0/25) in us-east-1a and techno-subnet-private-2 (35.10.12.0/25) in us-east-1b. These private subnets are intended for backend components such as Lambda functions attached to the VPC and the RDS database instance.

All subnets are assigned an IPv6 /64 block, and an Egress-only Internet Gateway is configured to allow outbound IPv6 traffic while preventing inbound connections from the internet. Separate route tables are maintained for public and private subnets to enforce proper traffic segregation—public subnets route internet-bound traffic through an Internet Gateway, while private subnets route outbound IPv4 traffic through a NAT Gateway (if configured) and IPv6 traffic through the egress-only gateway. This architecture ensures a secure, highly available, and well-segmented network environment suitable for a production-grade serverless system.

2. Security Groups

The security layer is enforced using Security Groups within Amazon VPC, ensuring controlled traffic flow between components inside the network.

The first security group, techno-sg-lambda, is attached to all Lambda functions running inside the VPC. This security group does not define any inbound rules, as Lambda functions do not require direct inbound traffic. Outbound traffic is allowed so that the functions can communicate with internal services such as the RDS database, VPC endpoints, and other AWS services.

The second security group, techno-sg-rds, is dedicated to the Amazon RDS instance. It allows inbound traffic exclusively on port 5432 (PostgreSQL) and only from techno-sg-lambda. This strict rule ensures that database access is restricted to authorized Lambda functions and cannot be accessed from public sources or other unintended resources.

The third security group, techno-sg-vpc-endpoint, is associated with Interface VPC Endpoints. It permits inbound HTTP (port 80) and HTTPS (port 443) traffic originating from the CIDR block 35.10.0.0/18. This configuration enables controlled connectivity to private AWS service endpoints while maintaining network isolation and adherence to the principle of least privilege.

3. VPC Endpoints

To ensure secure and private communication between services within the VPC, several VPC Endpoints are configured inside techno-vpc-serverless. These endpoints allow resources such as Lambda functions running in private subnets to access AWS services without traversing the public internet.

The techno-s3-endpoints endpoint is created for the service name com.amazonaws.us-east-1.s3 using the Gateway type. This gateway endpoint is associated with private route tables, enabling secure access to Amazon S3 for operations such as reading order data, storing logs, or accessing application assets. The techno-eventbridge-endpoints endpoint connects to com.amazonaws.us-east-1.events using the Interface type, allowing private connectivity to Amazon EventBridge. Similarly, the techno-steps-endpoints endpoint provides private access to com.amazonaws.us-east-1.states, enabling communication with AWS Step Functions. The techno-sns-endpoints endpoint connects to com.amazonaws.us-east-1.sns for secure integration with Amazon SNS, while the techno-secretsmanager-endpoints endpoint enables private access to com.amazonaws.us-east-1.secretsmanager, ensuring that credentials stored in AWS Secrets Manager can be retrieved without public exposure.

All interface endpoints are deployed within private subnets, reinforcing a fully private, secure, and internet-isolated serverless architecture.

3.2 S3 Bucket

3.2.1 S3 Bucket Order Documents

The primary bucket, **techno-orders-yourname-26**, is used to store order-related documents such as invoices, receipts, and supporting files. Versioning is **enabled** to protect against accidental overwrites or deletions, ensuring data recoverability. Server-side encryption is configured using **SSE-S3 (AES-256)** to automatically encrypt all objects at rest.

A lifecycle policy is implemented on the **Amazon S3** bucket to optimize long-term storage costs while maintaining data durability and accessibility. After objects have been stored for 30 days, they are automatically transitioned to the Intelligent-Tiering storage class. This allows S3 to dynamically move data between frequent and infrequent access tiers based on usage patterns, ensuring cost efficiency without performance trade-offs.

If the objects remain unused, they are further transitioned to the Glacier storage class after 120 days, significantly reducing storage costs for archival data that is rarely accessed. Finally, to enforce data retention policies and prevent indefinite storage growth, objects are automatically deleted after 365 days. This lifecycle strategy balances operational needs, compliance considerations, and cost optimization in a structured and automated manner.

The bucket policy restricts access to the designated Lambda execution role and the configured VPC Endpoint, preventing public exposure. Additionally, CORS is configured to allow requests originating from the deployed frontend hosted on AWS Amplify, enabling secure browser-based uploads and downloads.

3.2.2 S3 Bucket Application Logs

The second bucket, **techno-logs-yourname-2026**, is dedicated to storing structured application logs and exported monitoring data. To manage storage efficiency, lifecycle rules transition objects to **Intelligent-Tiering** after 14 days and automatically delete them after 120 days. This approach balances observability needs with cost optimization while maintaining compliance with data retention policies.

3.3 Secrets Manager & Parameter Store

The application implements secure configuration management using AWS Secrets Manager and Parameter Store. Sensitive database credentials are stored in Secrets Manager under the name techno-secret-rds-credentials. The secret is structured in JSON format containing the database host, database name (ordersdb), username (dbadmin), and password (TechnoCloud2026!). Storing credentials in JSON format simplifies programmatic retrieval and parsing within the application layer.

To enhance security and comply with best practices, automatic rotation is enabled with a 30-day rotation schedule. This ensures that database credentials are periodically regenerated and updated without manual intervention, reducing the risk of credential leakage or long-term exposure. By separating sensitive secrets from application code and infrastructure templates, the system enforces a secure, scalable, and production-ready secret management strategy.

3.4 Relational Database Service (RDS)

The relational database layer is provisioned using Amazon RDS with the DB identifier techno-rds-orders. The engine selected is PostgreSQL 15.x, ensuring compatibility with modern SQL features and long-term support. The instance runs on the db.t3.micro class, suitable for lightweight workloads and development-to-moderate production environments. Storage is configured at 20 GB using gp3, with encryption enabled through AWS Key Management Service (KMS) to protect data at rest.

For network isolation, the RDS instance is deployed inside techno-vpc-serverless, restricted to private subnets only, with public access disabled. Access control is enforced via a dedicated security group named techno-sg-rds, allowing database connectivity only from authorized services such as Lambda functions within the same VPC. The database created inside the instance is named ordersdb.

Database credentials are securely stored in techno-secret-rds-credentials using AWS Secrets Manager, preventing hardcoded credentials in application code or configuration files. Automated backups are enabled with a 7-day retention period, and the backup window is scheduled between 03:00–04:00 UTC to minimize operational impact. Additionally, deletion protection is enabled, ensuring that the database cannot be accidentally removed without explicit configuration changes, thereby safeguarding production data.

3.5 Lambda Functions

The system deploys seven serverless functions using **AWS Lambda** with the Python 3.11 runtime. In the advanced implementation, all functions are enhanced with **X-Ray tracing**, secure credential retrieval via **AWS Secrets Manager**, and structured logging in JSON format to ensure observability, traceability, and production-grade reliability.

The **techno-lambda-order-management** function is configured with 512 MB memory and a 30-second timeout. It supports X-Ray tracing, retrieves database credentials from Secrets Manager, implements database retry logic up to three attempts, supports soft delete operations, and provides order filtering by status.

The **techno-lambda-process-payment** function also runs with 512 MB memory and a 30-second timeout, featuring X-Ray tracing, idempotency control backed by **Amazon DynamoDB**, exponential backoff for retries, and strict payment amount validation to prevent inconsistencies.

The **techno-lambda-update-inventory** function is allocated 256 MB memory with a 45-second timeout. It integrates X-Ray tracing, Secrets Manager access, and implements pessimistic locking using FOR UPDATE queries to prevent race conditions, along with per-product stock threshold validation.

The **techno-lambda-send-notification** function (256 MB, 60-second timeout) handles EventBridge-triggered events, sends structured notification templates via **Amazon SNS**, and applies a three-time retry mechanism for reliability.

For reporting, the **techno-lambda-generate-report** function is provisioned with 1024 MB memory and a 120-second timeout to handle heavier workloads. It generates Excel reports with five sheets, creates a 24-hour pre-signed URL for secure download via **Amazon S3**, and sends a completion notification through SNS.

The **techno-lambda-init-db** function (512 MB, 300-second timeout) initializes the database schema with safe ALTER statements, creates PostgreSQL triggers, and seeds the database with ten sample products and five sample orders.

Running the techno-lambda-init-db with payload:

```
# Test Event for techno-lambda-init-db
{
   "insert_sample_data": true,
   "drop_existing": false
}

# Sample data entered:
# - 5 customers (CUST001–CUST005)
# - 10 products di 3 categories (Electronics, Accessories, Storage)
# - 5 orders with another status (completed, processing, pending, failed)
```

Lastly, the newly introduced **techno-lambda-health-check** function operates with 128 MB memory and a 10-second timeout. It performs system health validation by checking database connectivity (Amazon RDS), S3 availability, and **AWS Step Functions** execution status. This function is integrated as a lifecycle hook in **AWS CodeDeploy**, returning HTTP 503 when the system is unhealthy to prevent faulty deployments during Blue/Green releases.

Lambda Layer (techno-layer-dependencies)

A shared Lambda layer named **techno-layer-dependencies** is created to centralize common third-party libraries used across all functions running on **AWS Lambda**. This approach improves deployment efficiency, reduces package duplication, and simplifies version management across multiple Lambda functions.

The layer includes the following dependencies defined in requirements.txt:

```
psycopg2-binary
boto3
requests
pandas
openpyxl
aws-xray-sdk
```

By packaging these libraries into a reusable layer, each Lambda function remains lightweight and focuses only on business logic. When updates to dependencies are required, a new version of the layer can be published and attached to the functions without modifying the core application code. This promotes maintainability, consistency, and cleaner CI/CD workflows in a production-grade serverless environment.

3.5.1 Environment Variables Lambda

All credentials must **NOT** be stored directly in environment variables. Instead, secure access must be implemented using a **Secret ARN** retrieved from **AWS Secrets Manager**.

Each Lambda function deployed in **AWS Lambda** must reference the Secret ARN (for example, via an environment variable that stores only the ARN, not the credentials themselves). The function should then programmatically retrieve the secret value at runtime using the AWS SDK. This ensures that sensitive data such as database usernames, passwords, and host information are never hardcoded in the source code or exposed in plain-text configuration.

Developers must follow the implementation pattern already defined in the Lambda documentation within the repository. The repository provides the standardized method for securely fetching secrets, handling caching, and implementing fallback logic if necessary. By enforcing Secret ARN usage and eliminating direct credential storage in environment variables, the system adheres to security best practices and enterprise compliance standards.

3.5.2 Lambda Aliases

To enable Blue/Green deployments using **AWS CodeDeploy**, each function deployed in **AWS Lambda** must have a dedicated alias. CodeDeploy requires an alias (commonly named live) because traffic shifting occurs between Lambda versions through that alias rather than directly to $LATEST.

Before configuring CodeDeploy, the first step is to publish an initial immutable version of each Lambda function. After publishing version 1, an alias named **live** must be created and pointed to that version. For example, for the techno-lambda-order-management function, you first publish the version and then create an alias that references version 1. This establishes a stable production reference that CodeDeploy can manage during future deployments.

3.6 Simple Notification Service (SNS)

The notification layer is implemented using **Amazon SNS** with a Standard topic named **techno-sns-order-notifications**. This topic acts as the centralized messaging hub for system-wide notifications generated throughout the order processing workflow.

An email subscription is configured to **handi@seamolec.org**, ensuring that operational alerts and business notifications are delivered directly to stakeholders. The system publishes structured notification messages categorized into several types: **order_confirmation**, **payment_failed**, **low_stock**, **system_error**, **report_ready**, and **deployment**. These categories allow consistent message formatting and easier filtering or monitoring.

In the advanced implementation, the publishing Lambda functions implement a retry mechanism with up to **three attempts using exponential backoff** before marking the notification as failed. This approach increases reliability in case of transient service issues, helping guarantee message delivery while preventing excessive retry storms. Combined with structured logging and monitoring, this design ensures a resilient and production-ready notification system within the serverless architecture.

3.7 API Gateway

The API layer is implemented using Amazon API Gateway with the API name techno-api-orders. It is configured as a REST API with a Regional endpoint, ensuring low latency access within the deployed AWS region while maintaining scalability and high availability. To enforce secure communication, the API is configured with a TLS 1.3 policy, guaranteeing modern encryption standards for all inbound HTTPS requests.

The API is deployed to a dedicated stage named production, representing the live environment. For observability and distributed tracing, X-Ray tracing is enabled, allowing request flows to be monitored end-to-end. Additionally, access logging is activated and integrated with Amazon CloudWatch Log Groups, enabling detailed tracking of incoming requests, response status codes, latency, and client metadata for auditing and troubleshooting purposes.

To control and manage API consumption, a Usage Plan named techno-usage-plan is configured. The throttling policy enforces a rate limit of 1,000 requests per second with a burst capacity of 2,000 requests, ensuring protection against sudden traffic spikes. A monthly quota of 100,000 requests is applied to regulate overall usage. Access to the API is secured using an API key named techno-api-key, which must be included in client requests. Furthermore, a Request Validator is configured to validate both request body payloads and request parameters, ensuring that only properly structured and compliant requests are processed by the backend services.

Method	Resource	Lambda
GET	/orders	techno-lambda-order-management
POST	/orders	techno-lambda-order-management
GET	/orders/{id}	techno-lambda-order-management
PUT	/orders/{id}	techno-lambda-order-management
DELETE	/orders/{id}	techno-lambda-order-management
GET	/status/{id}	techno-lambda-order-management
GET	/customers	techno-lambda-order-management
GET	/products	techno-lambda-order-management
GET	/health	techno-lambda-health-check

4. Implementation without CloudFormation

4.1 Step Functions

State machine **techno-stepfunctions-order-workflow** is configured using the **Standard workflow** type in AWS Step Functions and uses the IAM role **LabRole** to grant the necessary permissions for invoking Lambda functions and handling execution flows. The workflow is designed to orchestrate an advanced order processing system, starting from order validation through payment processing, inventory updates, and final confirmation handling.

The execution begins at the **ValidateOrder** state, which is a Task state that invokes the Lambda function techno-lambda-order-management:live. This function receives parameters such as orderId, customerId, items, and totalAmount, and performs order validation. The result of this validation is stored in $.validationResult. The state is configured with a retry policy to automatically handle transient Lambda service errors, using exponential backoff with up to three attempts. If any unhandled error occurs, the workflow transitions to the **OrderFailed** state, and the error details are stored in $.errorInfo.

If validation succeeds, the workflow proceeds to the **ProcessPayment** state, another Task state that invokes the Lambda function techno-lambda-process-payment:live. This function processes the payment using the provided orderId and totalAmount, and the result is stored in $.paymentResult. After payment processing, the workflow transitions to a Choice state named **PaymentChoice**.

In the **PaymentChoice** state, the workflow evaluates the value of $.paymentResult.paymentStatus. If the status equals "success", the execution continues to the **UpdateInventory** state to adjust stock levels. If the payment is not successful, the workflow moves to the **PaymentFailed** state to handle the failure scenario.

Subsequent states such as **UpdateInventory**, **InventoryChoice**, and **SendConfirmation** manage inventory updates, validate stock availability, and send order confirmation notifications to the customer. Additional failure-handling states like **InventoryFailed** and **OrderFailed** ensure that any issue during payment, inventory, or validation is properly managed and logged. Overall, this state machine implements a resilient, fault-tolerant orchestration pattern that coordinates multiple Lambda functions in a structured and reliable order-processing workflow.

4.2 EventBridge

Within the event-driven architecture, **Amazon EventBridge** is configured with three dedicated rules to orchestrate automation and system responsiveness.

The first rule, **techno-eventbridge-daily-report**, is a scheduled rule that runs using the cron expression cron(59 23 * * ? *), which triggers at 23:59 UTC every day. Its target is the Lambda function **techno-lambda-generate-report**, responsible for generating daily operational reports. This ensures that reporting is executed automatically at the end of each day without manual intervention, supporting operational visibility and audit requirements.

The second rule, **techno-eventbridge-order-status**, is based on an event pattern rather than a schedule. It listens for custom application events where the source is defined as techno.order.system and the detail-type is InventoryLowStock. When such an event is published—typically emitted by a Lambda function within the order processing workflow—the rule routes the event to **techno-lambda-send-notification**. This enables near real-time notification handling whenever low inventory conditions are detected, reinforcing the system's reactive and loosely coupled design.

The third rule, **techno-eventbridge-low-stock**, uses a scheduled pattern defined by rate(1 hour). It triggers the **techno-lambda-update-inventory** function every hour to proactively check inventory levels. This periodic validation complements the event-driven low-stock detection mechanism, ensuring inventory consistency even if no triggering event occurs. Together, these three EventBridge rules combine scheduled automation and event-based routing to maintain system reliability, operational awareness, and responsiveness.

The techno-eventbridge-order-status rule in Amazon EventBridge is configured using an Event Pattern to capture specific custom events generated within the order processing system. Instead of relying on a scheduled trigger, this rule listens for real-time application events that match predefined criteria.

```
{
  "source": ["techno.order.system"],
  "detail-type": ["InventoryLowStock"]
}
```

4.3 CloudWatch

In Amazon CloudWatch, log monitoring is configured using multiple Log Groups with a retention period of 7 days to ensure observability while maintaining cost efficiency. Each Lambda function involved in the order processing architecture has its own dedicated Log Group under the /aws/lambda/ namespace.

These include

1. /aws/lambda/techno-lambda-order-management,
2. /aws/lambda/techno-lambda-process-payment,
3. /aws/lambda/techno-lambda-update-inventory,
4. /aws/lambda/techno-lambda-send-notification,
5. /aws/lambda/techno-lambda-generate-report,
6. /aws/lambda/techno-lambda-init-db, and
7. /aws/lambda/techno-lambda-health-check.

These Log Groups capture execution logs, errors, and performance details for each respective Lambda function.

In addition, the state machine execution logs for techno-stepfunctions-order-workflow are stored under /aws/states/techno-stepfunctions-order-workflow, enabling visibility into workflow transitions, task results, retries, and failures within AWS Step Functions. API request and execution logs are recorded in the Log Group API-Gateway-Execution-Logs_{rest-api-id}/production, providing detailed insights into request handling, integration responses, and potential errors within Amazon API Gateway.

Furthermore, deployment-related logs for the application are stored under /aws/codedeploy/techno-codedeploy-app, which captures deployment lifecycle events and traffic shifting activities managed by AWS CodeDeploy.
Overall, this logging configuration ensures comprehensive monitoring across Lambda functions, state machine workflows, API executions, and deployment processes, supporting troubleshooting, auditing, and performance analysis within the system.

4.3.1 CloudWatch Alarms

A comprehensive set of alarms is configured in **Amazon CloudWatch** to ensure proactive monitoring and automatic mitigation of potential system issues. These alarms continuously evaluate critical metrics across Lambda, API Gateway, Step Functions, and RDS components. When defined thresholds are exceeded, notifications are sent via **Amazon SNS**, and selected alarms are integrated with **AWS CodeDeploy** to automatically trigger rollback during faulty deployments.
For Lambda functions, an alarm monitors total errors and is triggered if errors exceed five occurrences within five minutes. Another alarm tracks the P99 duration metric and triggers if execution time exceeds 3000 milliseconds within three minutes, ensuring latency spikes are detected early.
At the API layer, two alarms monitor 4XX and 5XX error rates. Excessive 4XX errors indicate potential client-side misuse or integration problems, while 5XX errors signal backend instability. The 5XX alarm is configured as a rollback trigger in CodeDeploy to prevent unstable releases from impacting users.
For workflow reliability, an alarm monitors Step Functions execution failures and triggers if failures exceed three within ten minutes. Database health is monitored through an RDS CPU utilization alarm, which activates if CPU usage exceeds 80% over a five-minute period. This helps detect performance bottlenecks before they escalate.

4.3.2 CloudWatch Dashboard

A centralized operational dashboard named techno-dashboard-serverless is created in CloudWatch to provide real-time visibility into the entire serverless architecture. The dashboard aggregates key performance indicators from all critical components into a single pane of glass.

The first widget displays Lambda invocation counts, error rates, and P99 duration metrics across all seven functions, allowing quick identification of abnormal behavior. The second widget focuses on API Gateway metrics, including request count, 4XX and 5XX errors, and latency trends. The third widget tracks Step Functions executions, showing the number of started, succeeded, and failed workflows.

The fourth widget monitors database performance metrics such as CPU utilization, active connections, and read/write IOPS from Amazon RDS. Finally, a deployment widget provides visibility into CodeDeploy deployment status and traffic shifting progress, ensuring that release activities can be monitored in real time.
This dashboard enables engineering and operations teams to quickly assess system health and respond to anomalies efficiently.

4.3.3 AWS X-Ray

Distributed tracing is implemented using AWS X-Ray to achieve end-to-end visibility across the serverless workflow. Active tracing is enabled on all seven Lambda functions as well as on the production stage of API Gateway. This configuration allows

each request to be traced from the API entry point through Lambda execution, downstream service calls, database queries, and notification services.

An X-Ray group named techno-xray-group is configured with a filter expression targeting the primary order management service. This grouping allows focused monitoring and performance analysis of the core business workflow.

Additionally, X-Ray Insights is enabled to automatically detect anomalies such as increased latency, elevated fault rates, or unusual traffic patterns. By leveraging automated anomaly detection, the system can proactively identify performance degradation or emerging issues before they significantly impact users.

Together, CloudWatch and X-Ray provide a robust observability framework that supports monitoring, automated rollback protection, and deep performance diagnostics across the entire serverless environment.

5. AWS CodeDeploy — Blue/Green Deployment

AWS CodeDeploy implements a **Blue/Green** deployment strategy for AWS Lambda functions by leveraging Lambda versioning and weighted aliases. Each new deployment publishes a new Lambda version and gradually shifts traffic from the current production version to the newly deployed version. During this transition, Amazon CloudWatch alarms are continuously evaluated to determine whether the deployment should proceed or automatically roll back. This approach minimizes risk by validating the new version under real production traffic before full rollout.

5.1 Blue/Green concept for Lambda

In this architecture, the Blue environment represents the currently active Lambda version serving 100% of production traffic. The alias live points to this stable version.

The Green environment represents the newly deployed Lambda version. Initially, only 10% of traffic is shifted to the new version using a weighted alias configuration.

The Canary period lasts five minutes, using the predefined deployment configuration LambdaCanary10Percent5Minutes. During this monitoring window, system health metrics such as Lambda errors, latency (P99), and API 5XX errors are evaluated.

If no alarms are triggered, traffic is automatically shifted to 100% Green. However, if any configured CloudWatch alarm enters the ALARM state, CodeDeploy immediately performs an automatic rollback by redirecting the alias back to the Blue version.

This controlled traffic shifting ensures safe and observable production releases.

5.2 CodeDeploy Application Configuration

The deployment configuration is defined within AWS CodeDeploy as a Lambda-based application named techno-codedeploy-app. This application uses the Lambda compute platform, enabling version-based deployments with traffic shifting capabilities.

A dedicated deployment group called techno-codedeploy-group is configured to manage how updates are released to production. The deployment process operates under the IAM role LabRole, which grants the necessary permissions to update Lambda aliases, monitor alarms, and execute lifecycle hooks.

The deployment strategy uses the predefined configuration CodeDeployDefault.LambdaCanary10Percent5Minutes, meaning that when a new Lambda version is deployed, only 10% of production traffic is shifted to the new version for a five-minute evaluation period. During this canary phase, system health is continuously monitored before allowing full traffic migration.

To ensure safe deployments, rollback alarms are explicitly attached to the deployment group. These alarms monitor Lambda error rates, P99 latency, and API 5XX errors through Amazon CloudWatch. If any of these alarms enter the ALARM state during deployment, CodeDeploy automatically rolls traffic back to the previous stable version.

Additionally, lifecycle validation hooks are implemented using the techno-lambda-health-check function. A pre-traffic hook validates critical dependencies such as database and storage connectivity before traffic is shifted, while a post-traffic hook verifies system stability after the shift is completed.

This configuration ensures that every deployment is gradual, observable, automatically validated, and capable of immediate rollback, providing a robust and production-ready release process.

6. Implement **CI/CD Pipeline — GitHub Actions**

The CI/CD pipeline is implemented using GitHub Actions to automate testing, deployment, and notification processes for the application. This pipeline ensures that every change pushed to the repository is validated and deployed in a structured and controlled manner.

GitHub Secrets Configuration

To enable secure authentication with AWS, several repository secrets must be configured in GitHub. These include AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_SESSION_TOKEN, all obtained from AWS Academy Learner Lab. Because the Learner Lab credentials are temporary, these secret values must be updated each time a new lab session is started. Storing them as GitHub Secrets ensures that sensitive credentials are not exposed in the workflow file.

Pipeline Structure (5 Jobs)

The pipeline consists of five sequential jobs with defined dependencies and execution conditions:

1. **track-and-notify-owner**
   This job runs unconditionally at the beginning of the workflow. It detects whether the repository has been forked, retrieves the user's email address, and sends a notification via SNS if a fork is detected. This ensures visibility and governance over repository usage.
2. **test**
   This job depends on the first job and runs only if the parameter skip_tests is not set to true. It performs syntax validation for all seven Lambda functions and runs flake8 linting checks to ensure code quality and consistency before deployment.
3. **deploy-lambda**
   This job executes only on the **master** branch. It updates the Lambda function code,

publishes a new version, and triggers a deployment process using CodeDeploy. This ensures versioned, controlled updates for backend services.

4. **deploy-frontend**
   Also restricted to the master branch, this job validates frontend files, triggers a new build in Amplify, and monitors the build process until completion. This guarantees that the frontend is successfully deployed after backend updates.
5. **notify**
   The final job runs after both backend and frontend deployment jobs complete and always executes on the **master** branch. It sends a deployment report via SNS, summarizing the pipeline result and providing confirmation of success or failure.

Overall, this GitHub Actions CI/CD pipeline enforces automated testing, controlled backend and frontend deployment, and structured notification handling, ensuring reliability, traceability, and operational transparency across the system.

7. Deploy to Amplify

To deploy the application to AWS Amplify, create a new app in AWS Amplify named techno-amplify-order-app and connect it to your GitHub repository at https://github.com/[username]/wakawaka (using your forked repository). Select the master branch as the deployment source. Since the project uses Static HTML/CSS/JavaScript, no specific framework configuration (such as React or Vue) is required. Ensure that the build configuration file amplify.yml is located in the root directory of the repository. In the build settings, specify the artifacts directory as the frontend/ folder, as it contains the main application files, including index.html and app.js. With this configuration, AWS Amplify will execute the instructions defined in amplify.yml and publish the application from the frontend/ directory.

In addition, configure the required environment variables in the Amplify Console. Navigate to App settings → Environment variables → Add variable, then add the following variables: API_ENDPOINT with the value https://{rest-api-id}.execute-api.us-east-1.amazonaws.com/production, API_KEY with the value obtained from techno-api-key (retrieved via API Gateway → API Keys → Show), and AWS_REGION with the value us-east-1. These environment variables ensure that the frontend application can properly communicate with the deployed backend API.

8. Testing & Verification

8.1 API Testing

Test the whole endpoint using Postman or curl

```
API_URL="https://YOUR_API_ID.execute-api.us-east-
1.amazonaws.com/production"
API_KEY="YOUR_API_KEY_VALUE"
# 1. Health Check
curl -H "x-api-key: $API_KEY" $API_URL/health
# Expected: {"status": "healthy", "checks": {...}}

# 2. Create Order → trigger Step Functions
curl -X POST $API_URL/orders -H "x-api-key: $API_KEY" \
```

```
    -H "Content-Type: application/json" \
    -d
'{"customer_id":"CUST001","items":[{"product_id":"PROD001","quantity":1}]}'
# Expected 201: {"order_id":"...", "execution_arn":"..."}

# 3. Check Step Functions status
curl -H "x-api-key: $API_KEY" "$API_URL/status/{execution_arn}"

# 4. List orders with filter
curl -H "x-api-key: $API_KEY" "$API_URL/orders?status=completed&limit=5"
```

8.2 Workflow Testing

The Step Functions state machine is tested by providing input from the step_function/order.json file. The testing process verifies three main execution paths: the normal workflow where payment is successful, the scenario where payment fails, and the scenario where inventory validation fails. Each execution is monitored in the AWS Step Functions console to ensure that state transitions, Choice conditions, retries, and failure handlers behave according to the defined workflow logic.

8.3 CodeDeploy Testing

To test deployment automation, a minor change is pushed to one of the Lambda functions. The GitHub Actions pipeline is then monitored to confirm that all six jobs execute sequentially without errors. In the AWS CodeDeploy Console, the deployment process is verified to ensure that it starts correctly and applies a canary deployment strategy, shifting 10% of traffic for five minutes before routing 100% of traffic to the new version. After completion, confirmation is made that the Lambda alias **live** points to the newly published version.

8.4 Blue/Green Rollback Testing

To validate rollback mechanisms, an error is intentionally injected into the Lambda function (for example, forcing it to return HTTP 500 for all requests), and the change is pushed to the **master** branch. Once CodeDeploy initiates the canary deployment, monitoring is performed to confirm that the CloudWatch alarm **techno-alarm-api-5xx** is triggered within five minutes. Upon alarm activation, CodeDeploy should automatically roll back the deployment to the previous stable version. After rollback, the API is tested again to ensure that it functions normally.

8.5 EventBridge Testing

The scheduled reporting workflow is tested by manually triggering the rule **techno-eventbridge-daily-report** using the Test Rule feature in the Amazon EventBridge Console. Verification includes confirming that the Lambda function **techno-lambda-generate-report** is invoked successfully, an Excel file is generated in the S3 bucket under the reports/ directory, and an SNS notification is sent containing a pre-signed URL for downloading the report.

8.6 Frontend Testing

Frontend validation is performed by accessing the deployed application URL from AWS Amplify. An order is created through the form interface, and the dashboard

statistics are checked to confirm real-time updates. The workflow status is validated to ensure proper integration with backend services, and report downloads are tested from the Reports section to confirm end-to-end functionality.

8.7 Integration Testing

End-to-end integration testing begins by creating an order from the frontend interface. The execution is then monitored in the Step Functions console to verify correct orchestration. Inventory reduction is validated directly in the database, email notifications are confirmed as received, and distributed tracing is examined through AWS X-Ray Service Map to visualize interactions among services such as Lambda, API Gateway, Step Functions, and the database. This comprehensive testing ensures that all system components operate cohesively within the cloud architecture.